부록 · Docker 로 DB 설치

Docker MongoDB8 초보자 실습 교본

Docker MongoDB 8 초보자 실습 교본

Rocky Linux + Docker + MongoDB 8 + mongosh 설치 · 실행 · 사용자 · 컬렉션 · CRUD · 검색 · 인덱스 · 100개 명령어

실습 항목	기준값
컨테이너 이름	wiz-mongodb
MongoDB 이미지	mongo:8
호스트 포트	27017
관리자 계정	root (admin DB)
실습 DB	wizdb
실습 사용자	devuser
실습 컬렉션	users

※ 비밀번호는 문서에 고정값으로 저장하지 말고 실제 서버에서 안전한 값으로 설정하는 것을 권장함.

1. MongoDB를 SQL DB와 비교해서 이해하기

관계형 DB	MongoDB	설명
Database	Database	데이터베이스
Table	Collection	문서(Document)를 담는 집합
Row	Document	한 건의 데이터
Column	Field	문서 내부의 항목
Primary Key	_id	MongoDB가 기본적으로 자동 생성
INSERT	insertOne / insertMany	데이터 입력
SELECT	find / findOne	데이터 조회
UPDATE	updateOne / updateMany	데이터 수정
DELETE	deleteOne / deleteMany	데이터 삭제

2. Docker MongoDB 기본 구성

아래 docker-compose.yml 예제는 MongoDB 데이터를 ./data에 영구 보관하고, 서버의 27017 포트를 컨테이너의 27017 포트에 연결함.

```yaml
services:
  mongodb:
    image: mongo:8
    container_name: wiz-mongodb
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: "안전한_관리자_비밀번호"
    volumes:
      - ./data:/data/db
```

```bash
cd ~/docker/mongodb
docker compose up -d
docker ps
docker logs --tail 30 wiz-mongodb
```

3. MongoDB 접속과 사용자 만들기

관리자(root)는 admin DB에 인증하고, 응용프로그램용 devuser 사용자는 wizdb에 readWrite 권한으로 생성하는 방식이 이해하기 쉬움.

```bash
docker exec -it wiz-mongodb mongosh -u root -p --authenticationDatabase admin
```

```javascript
use wizdb
db.createUser({
  user: "devuser",
  pwd: "여기에_안전한_비밀번호",
  roles: [
    { role: "readWrite", db: "wizdb" }
  ]
})
show users
```

```bash
docker exec -it wiz-mongodb mongosh -u devuser -p --authenticationDatabase wizdb wizdb
```

비밀번호 변경은 사용자가 만들어진 DB에서 실행함.

```javascript
use wizdb
db.changeUserPassword("devuser", "새로운_안전한_비밀번호")
```

4. users 컬렉션 만들기

```javascript
use wizdb
db.createCollection("users")
show collections
```

MongoDB에서는 컬렉션을 명시적으로 만들지 않고 insertOne()을 처음 실행해도 자동 생성될 수 있으나, 학습 단계에서는 createCollection()으로 직접 만드는 것이 구조를 이해하기 좋음.

5. CREATE - 데이터 입력

5.1 한 명 입력

```javascript
db.users.insertOne({
  user_id: 1,
  username: "hong",
  name: "홍길동",
  age: 30,
  gender: "남",
  phone: "010-1111-1111",
  email: "hong@test.com",
  address: "서울시 강남구",
  job: "개발자",
  salary: 5000000,
  active: true,
  reg_date: new Date()
})
```

5.2 여러 명 입력

```javascript
db.users.insertMany([
  { user_id: 2, username: "kim",  name: "김철수", age: 25, job: "디자이너", salary: 3500000 },
  { user_id: 3, username: "lee",  name: "이영희", age: 28, job: "회계사",   salary: 4200000 },
  { user_id: 4, username: "park", name: "박민수", age: 35, job: "교사",     salary: 4500000 },
  { user_id: 5, username: "kang", name: "강감찬", age: 40, job: "회사원",   salary: 4800000 }
])
```

6. READ - 데이터 조회

```javascript
db.users.find()
db.users.findOne({ username: "kim" })
db.users.find({ age: { $gte: 30 } })
db.users.find({ salary: { $gte: 4500000 } })
```

원하는 필드만 깔끔하게 출력하려면 Projection을 사용함. 1은 표시, 0은 제외를 의미함.

```javascript
db.users.find(
  {},
  { _id: 0, name: 1, age: 1, job: 1 }
)
```

```javascript
db.users.findOne(
  { name: "홍길동" },
  { _id: 0, name: 1, phone: 1, email: 1 }
)
```

7. UPDATE - 데이터 수정

```javascript
db.users.updateOne(
  { username: "kim" },
  { $set: { age: 26 } }
)
```

```javascript
db.users.updateOne(
  { username: "kim" },
  { $inc: { salary: 100000 } }
)
```

```javascript
db.users.updateMany(
  { age: { $gte: 30 } },
  { $set: { group: "30+" } }
)
```

8. DELETE - 데이터 삭제

```javascript
db.users.deleteOne({ username: "kim" })
db.users.deleteMany({ active: false })
```

주의: 다음 명령은 users의 모든 문서를 삭제함.

```javascript
db.users.deleteMany({})
```

다음 명령은 컬렉션 자체를 삭제함.

```javascript
db.users.drop()
```

9. 검색, 정렬, 개수, 인덱스

```javascript
db.users.find().sort({ age: 1 })
db.users.find().sort({ salary: -1 }).limit(3)
db.users.countDocuments()
db.users.countDocuments({ age: { $gte: 30 } })
db.users.createIndex({ username: 1 }, { unique: true })
db.users.getIndexes()
```

10. CRUD 핵심 암기표

구분	대표 명령	의미
CREATE	db.users.insertOne({...})	한 건 입력
CREATE	db.users.insertMany([...])	여러 건 입력
READ	db.users.find()	전체 조회
READ	db.users.findOne({name:"홍길동"})	한 건 조회
UPDATE	db.users.updateOne({...}, {$set:{...}})	한 건 수정
DELETE	db.users.deleteOne({...})	한 건 삭제

11. MongoDB 실무·학습용 명령어 100개

명령은 Docker 셸 명령과 mongosh 명령을 함께 정리함. 삭제 명령은 실습 DB에서만 주의해서 사용할 것.

번호	분류	목적	명령어
1	Docker	컨테이너 실행 확인	docker ps
2	Docker	모든 컨테이너 확인	docker ps -a
3	Docker	MongoDB 시작	docker start wiz-mongodb
4	Docker	MongoDB 중지	docker stop wiz-mongodb
5	Docker	MongoDB 재시작	docker restart wiz-mongodb
6	Docker	MongoDB 로그	docker logs wiz-mongodb
7	Docker	최근 로그 30줄	docker logs --tail 30 wiz-mongodb
8	Docker	Compose 시작	docker compose up -d
9	Docker	Compose 종료	docker compose down
10	접속	root 접속	docker exec -it wiz-mongodb mongosh -u root -p --authenticationDatabase admin
11	접속	devuser 접속	docker exec -it wiz-mongodb mongosh -u devuser -p --authenticationDatabase wizdb wizdb
12	DB	DB 목록	show dbs
13	DB	현재 DB	db
14	DB	wizdb 선택	use wizdb
15	DB	컬렉션 목록	show collections
16	DB	users 생성	db.createCollection("users")
17	DB	컬렉션 통계	db.users.stats()
18	DB	DB 통계	db.stats()
19	DB	users 컬렉션 삭제	db.users.drop()
20	DB	현재 DB 삭제	db.dropDatabase()
21	입력	한 건 입력	db.users.insertOne({name:"홍길동"})
22	입력	여러 건 입력	db.users.insertMany([{name:"김철수"},{name:"이영희"}])
23	입력	숫자 입력	db.users.insertOne({age:30})
24	입력	Boolean 입력	db.users.insertOne({active:true})
25	입력	날짜 입력	db.users.insertOne({reg_date:new Date()})
26	입력	배열 입력	db.users.insertOne({skills:["Java","Kotlin"]})
27	입력	중첩 객체 입력	db.users.insertOne({address:{city:"서울"}})
28	조회	전체 조회	db.users.find()
29	조회	한 건 조회	db.users.findOne()
30	조회	이름 검색	db.users.find({name:"홍길동"})
31	조회	username 검색	db.users.find({username:"hong"})
32	조회	나이 30	db.users.find({age:30})
33	조회	30세 이상	db.users.find({age:{$gte:30}})

번호	분류	목적	명령어
34	조회	30세 초과	db.users.find({age:{$gt:30}})
35	조회	30세 이하	db.users.find({age:{$lte:30}})
36	조회	30세 미만	db.users.find({age:{$lt:30}})
37	조회	30세 제외	db.users.find({age:{$ne:30}})
38	조회	25·30세	db.users.find({age:{$in:[25,30]}})
39	조회	25·30세 제외	db.users.find({age:{$nin:[25,30]}})
40	조회	AND 검색	db.users.find({age:30,gender:"남"})
41	조회	명시적 AND	db.users.find({$and:[{age:30},{gender:"남"}]})
42	조회	OR 검색	db.users.find({$or:[{age:25},{age:30}]})
43	조회	이름만 표시	db.users.find({}, {_id:0,name:1})
44	조회	이름·나이 표시	db.users.find({}, {_id:0,name:1,age:1})
45	조회	이름·직업 표시	db.users.find({}, {_id:0,name:1,job:1})
46	조회	_id 제외	db.users.find({}, {_id:0})
47	정렬	나이 오름차순	db.users.find().sort({age:1})
48	정렬	나이 내림차순	db.users.find().sort({age:-1})
49	정렬	급여 내림차순	db.users.find().sort({salary:-1})
50	조회	5건 제한	db.users.find().limit(5)
51	조회	5건 건너뛰기	db.users.find().skip(5)
52	정렬	급여 상위 3명	db.users.find().sort({salary:-1}).limit(3)
53	집계	전체 건수	db.users.countDocuments()
54	집계	남성 회원 수	db.users.countDocuments({gender:"남"})
55	집계	30세 이상 수	db.users.countDocuments({age:{$gte:30}})
56	수정	나이 변경	db.users.updateOne({username:"hong"},{$set:{age:31}})
57	수정	직업 변경	db.users.updateOne({username:"hong"},{$set:{job:"교수"}})
58	수정	급여 변경	db.users.updateOne({username:"hong"},{$set:{salary:5500000}})
59	수정	여러 필드 변경	db.users.updateOne({username:"hong"},{$set:{age:31,job:"교수"}})
60	수정	급여 증가	db.users.updateOne({username:"hong"},{$inc:{salary:100000}})
61	수정	나이 1 증가	db.users.updateOne({username:"hong"},{$inc:{age:1}})
62	수정	phone 필드 삭제	db.users.updateOne({username:"hong"},{$unset:{phone:""}})
63	수정	여러 문서 수정	db.users.updateMany({gender:"남"},{$set:{active:true}})
64	수정	조건부 여러 건	db.users.updateMany({age:{$gte:30}},{$set:{group:"30+"}})
65	수정	Upsert	db.users.updateOne({username:"new"},{$set:{name:"신규"}},{upsert:true})
66	삭제	한 건 삭제	db.users.deleteOne({username:"hong"})

번호	분류	목적	명령어
67	삭제	이름으로 삭제	db.users.deleteOne({name:"홍길동"})
68	삭제	비활성 회원 삭제	db.users.deleteMany({active:false})
69	삭제	20세 미만 삭제	db.users.deleteMany({age:{$lt:20}})
70	삭제	모든 문서 삭제	db.users.deleteMany({})
71	인덱스	username 인덱스	db.users.createIndex({username:1})
72	인덱스	username UNIQUE	db.users.createIndex({username:1},{unique:true})
73	인덱스	email UNIQUE	db.users.createIndex({email:1},{unique:true})
74	인덱스	복합 인덱스	db.users.createIndex({name:1,age:1})
75	인덱스	인덱스 목록	db.users.getIndexes()
76	인덱스	username 인덱스 삭제	db.users.dropIndex("username_1")
77	인덱스	보조 인덱스 전체 삭제	db.users.dropIndexes()
78	검색	이름 정규식	db.users.find({name:/김/})
79	검색	대소문자 무시	db.users.find({username:/hong/i})
80	검색	email 필드 존재	db.users.find({email:{$exists:true}})
81	배열	배열 값 검색	db.users.find({skills:"Kotlin"})
82	배열	배열 값 추가	db.users.updateOne({username:"hong"},{$push:{skills:"Rust"}})
83	배열	중복 없이 추가	db.users.updateOne({username:"hong"},{$addToSet:{skills:"Java"}})
84	배열	배열 값 제거	db.users.updateOne({username:"hong"},{$pull:{skills:"Rust"}})
85	집계	평균 급여	db.users.aggregate([{$group:{_id:null,avg:{$avg:"$salary"}}}])
86	집계	최고 급여	db.users.aggregate([{$group:{_id:null,max:{$max:"$salary"}}}])
87	집계	최저 급여	db.users.aggregate([{$group:{_id:null,min:{$min:"$salary"}}}])
88	집계	급여 합계	db.users.aggregate([{$group:{_id:null,total:{$sum:"$salary"}}}])
89	집계	직업별 인원	db.users.aggregate([{$group:{_id:"$job",count:{$sum:1}}}])
90	집계	성별 인원	db.users.aggregate([{$group:{_id:"$gender",count:{$sum:1}}}])
91	사용자	사용자 목록	show users
92	사용자	devuser 조회	db.getUser("devuser")
93	사용자	비밀번호 변경	db.changeUserPassword("devuser","새비밀번호")
94	사용자	devuser 삭제	db.dropUser("devuser")
95	관리	admin DB	use admin
96	관리	서버 상태	db.serverStatus()
97	관리	MongoDB 버전	db.version()
98	관리	현재 연결 정보	db.getMongo()
99	도움말	mongosh 도움말	help

번호	분류	목적	명령어
100	종료	mongosh 종료	exit

12. 초보자가 꼭 기억할 10개

```javascript
use wizdb
show dbs
show collections
db.createCollection("users")
db.users.insertOne({...})
db.users.insertMany([...])
db.users.find()
db.users.findOne({name:"홍길동"})
db.users.updateOne({name:"홍길동"}, {$set:{age:31}})
db.users.deleteOne({name:"홍길동"})
```

13. 실습 시 주의사항

- docker compose down -v 는 볼륨 삭제로 이어질 수 있으므로 데이터가 필요한 서버에서는 함부로 사용하지 않음.
- db.users.deleteMany({}) 는 컬렉션의 모든 문서를 삭제하므로 조건을 먼저 find()로 확인한 뒤 실행함.
- db.dropDatabase() 는 현재 선택된 데이터베이스 전체를 삭제하므로 실습 DB가 맞는지 db 명령으로 먼저 확인함.
- MongoDB 사용자는 생성된 인증 DB가 중요함. devuser이 wizdb에 생성되었다면 --authenticationDatabase wizdb 로 접속함.
- 외부 접속을 허용할 때는 27017 포트를 인터넷 전체에 무방비로 공개하지 말고 방화벽과 인증을 함께 적용함.
